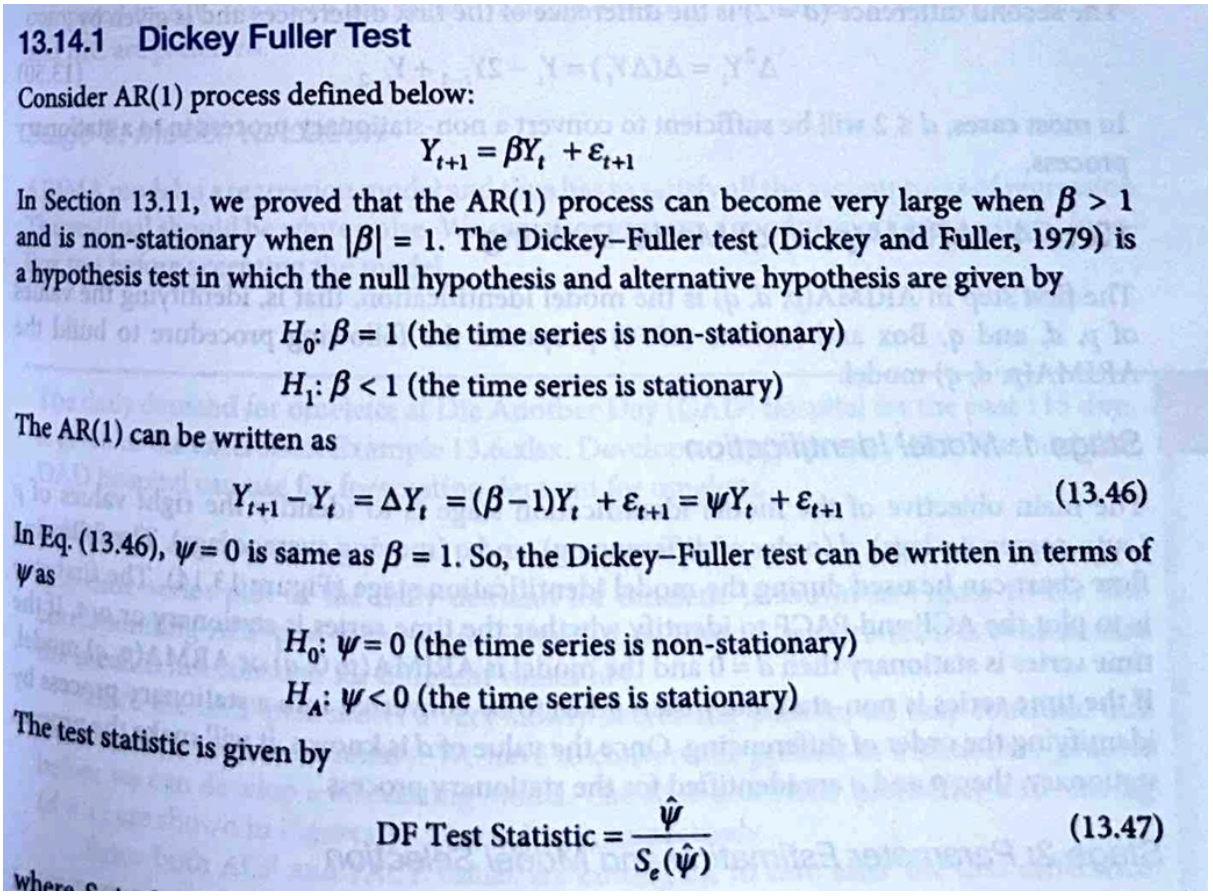
Dickey-Fuller test

1. Formulating the Hypotheses



13.14.1 Dickey Fuller Test

Consider AR(1) process defined below:

$$Y_{t+1} = \beta Y_t + \varepsilon_{t+1}$$

In Section 13.11, we proved that the AR(1) process can become very large when $\beta > 1$ and is non-stationary when $|\beta| = 1$. The Dickey–Fuller test (Dickey and Fuller, 1979) is a hypothesis test in which the null hypothesis and alternative hypothesis are given by

H_0: $\beta = 1$ (the time series is non-stationary)

H_1: $\beta < 1$ (the time series is stationary)

The AR(1) can be written as

$$Y_{t+1} - Y_t = \Delta Y_t = (\beta - 1)Y_t + \varepsilon_{t+1} = \psi Y_t + \varepsilon_{t+1} \quad (13.46)$$

In Eq. (13.46), $\psi = 0$ is same as $\beta = 1$. So, the Dickey–Fuller test can be written in terms of ψ as

H_0: $\psi = 0$ (the time series is non-stationary)

H_A: $\psi < 0$ (the time series is stationary)

The test statistic is given by

$$\text{DF Test Statistic} = \frac{\hat{\psi}}{S_e(\hat{\psi})} \quad (13.47)$$

Where

- **Null Hypothesis (H_0)**: The time series has a unit root (i.e., it is non-stationary).
- **Alternative Hypothesis (H_1)**: The time series does not have a unit root (i.e., it is stationary).

2. Transforming the Data

- The test begins with a regression of the time series on its lagged values to identify whether there's a unit root. The basic form is:

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \delta \Delta y_{t-1} + \epsilon_t$$

Where: - Δy_t is the change (first difference) of the time series. - y_{t-1} is the lagged value of the series. - α is a constant (drift). - βt represents a trend component (optional). - $\delta \Delta y_{t-1}$ includes lagged changes to account for autocorrelation. - ϵ_t is the error term.

3. Choosing the Test Version

There are three common forms of the Dickey-Fuller test:

- **No constant, no trend**: Only checks the series for random walk behavior.
- **With constant (drift)**: Includes a constant term to account for mean shifts in the data.
- **With constant and trend**: Includes both a constant and a trend to test for non-stationarity along with a deterministic trend.

4. Estimating Parameters

- Using regression, the coefficients of the equation are estimated. The key parameter of interest is γ\gamma, which indicates the presence of a unit root.
- For the null hypothesis H0H₀, γ=0\gamma = 0, meaning the series is non-stationary.

5. Performing the Test

- Calculate the test statistic for γ\gamma and compare it to the critical values provided in the Dickey-Fuller tables.
- The critical values depend on the size of the sample and the type of test (no constant, constant, or constant + trend).

6. Interpreting the Results

- If the test statistic is **less than the critical value**, reject the null hypothesis (H0H₀). This suggests the series is stationary.
- If the test statistic is **greater than or equal to the critical value**, fail to reject H0H₀. This implies the series is non-stationary.

7. Iterating if Necessary

- If the series is non-stationary, you can take differences (e.g., first or second differences) and reapply the test until stationarity is achieved. This process is often required for making the data suitable for time series modeling.

Augmented Dickey-Fuller (ADF) test

The Augmented Dickey-Fuller (ADF) test is an extension of the original Dickey-Fuller test, designed to handle more complex data scenarios. Here's how it differs:

1. Accounts for Autocorrelation

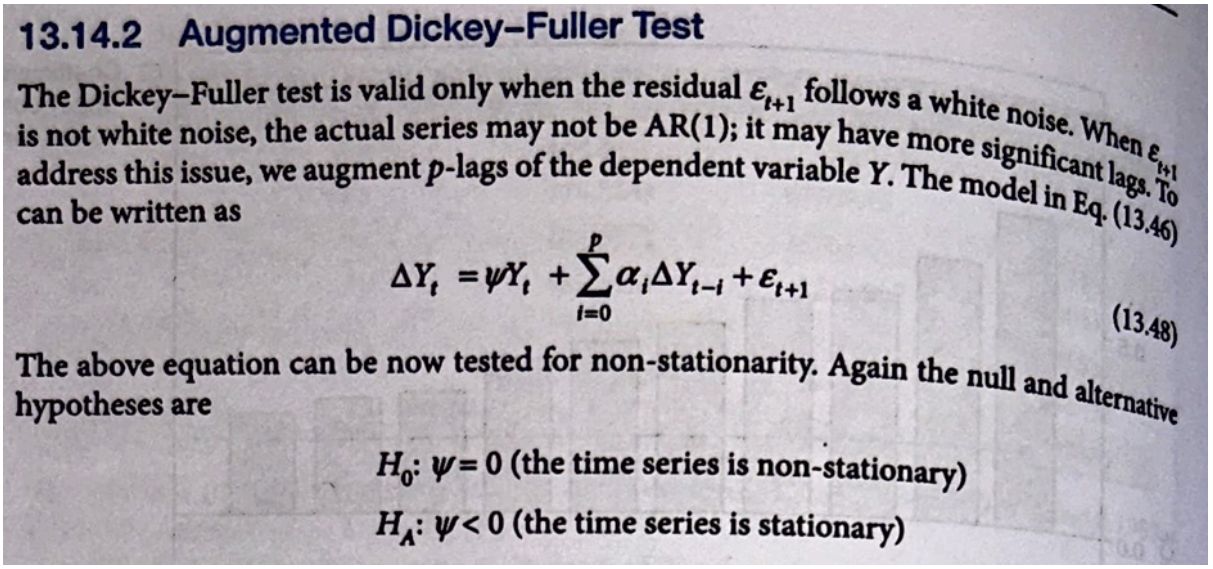
- **Dickey-Fuller Test**: Assumes no autocorrelation in the error terms. If autocorrelation exists, it may lead to incorrect results.
- **ADF Test**: Adds lagged differences of the dependent variable to the regression equation to account for autocorrelation in the time series. This makes the ADF test more robust for real-world data, where autocorrelation is common.

2. Modified Regression Equation

- The ADF test uses the following equation:

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^{p} \delta_i \Delta y_{t-i} + \epsilon_t$$

- $\sum_{i=1}^{p} \delta_i \Delta y_{t-i}$: This term introduces lagged differences of the series (Δy_{t-i}) to remove autocorrelation.
- p: Number of lagged differences included (chosen based on criteria like the Akaike Information Criterion).



13.14.2 Augmented Dickey–Fuller Test

The Dickey–Fuller test is valid only when the residual ε_{t+1} follows a white noise. When ε_{t+1} is not white noise, the actual series may not be AR(1); it may have more significant lags. To address this issue, we augment p-lags of the dependent variable Y. The model in Eq. (13.46) can be written as

$$\Delta Y_t = \psi Y_t + \sum_{i=0}^{p} \alpha_i \Delta Y_{t-i} + \varepsilon_{t+1} \tag{13.48}$$

The above equation can be now tested for non-stationarity. Again the null and alternative hypotheses are

H_0: $\psi = 0$ (the time series is non-stationary)

H_A: $\psi < 0$ (the time series is stationary)

Augmented Dickey-Fuller (ADF) test step-by-step guide

1. Install Necessary Libraries

Make sure you have `statsmodels` installed. You can do this via pip:

bash

```
pip install statsmodels
```

2. Import Libraries

Import the required libraries for data manipulation and statistical analysis:

python

```
import numpy as np
import pandas as pd
from statsmodels.tsa.stattools import adfuller
```

3. Prepare Your Data

You need a time series dataset. For example, if you have a pandas DataFrame `df` with a time series column `data`:

python

```
# Example time series data
data = [2.3, 2.5, 2.7, 2.6, 2.8, 3.0, 3.2, 3.3, 3.1, 3.4]
df = pd.DataFrame(data, columns=['data'])
```

4. Perform the ADF Test

Use the `adfuller()` function from `statsmodels.tsa.stattools`. Here's how to perform the test:

python

```
# Apply the Augmented Dickey-Fuller test
result = adfuller(df['data'])

# Extract the results
adf_statistic = result[0]
p_value = result[1]
```

```
critical_values = result[4]

# Print results
print("ADF Statistic:", adf_statistic)
print("p-value:", p_value)
print("Critical Values:", critical_values)
```

5. Interpret the Results

- **ADF Statistic**: If this is smaller than the critical values (at 1%, 5%, or 10%), you can reject the null hypothesis (the series is stationary).
- **p-value**: If this is less than 0.05, the null hypothesis is rejected at a 95% confidence level, suggesting the series is stationary.

Example Output

For instance, the output might look like this:

```
ADF Statistic: -3.5721
p-value: 0.0124
Critical Values: {'1%': -4.045, '5%': -3.451, '10%': -3.151}
```

If the ADF statistic is less than the critical value at, say, 5%, and the p-value is below 0.05, you conclude that the time series is stationary.

Seasonal ARIMA

Time series can be seasonal. What does this mean? It means that our data exhibits a season or cycle that regularly repeats, for example, weekly, monthly or annually.

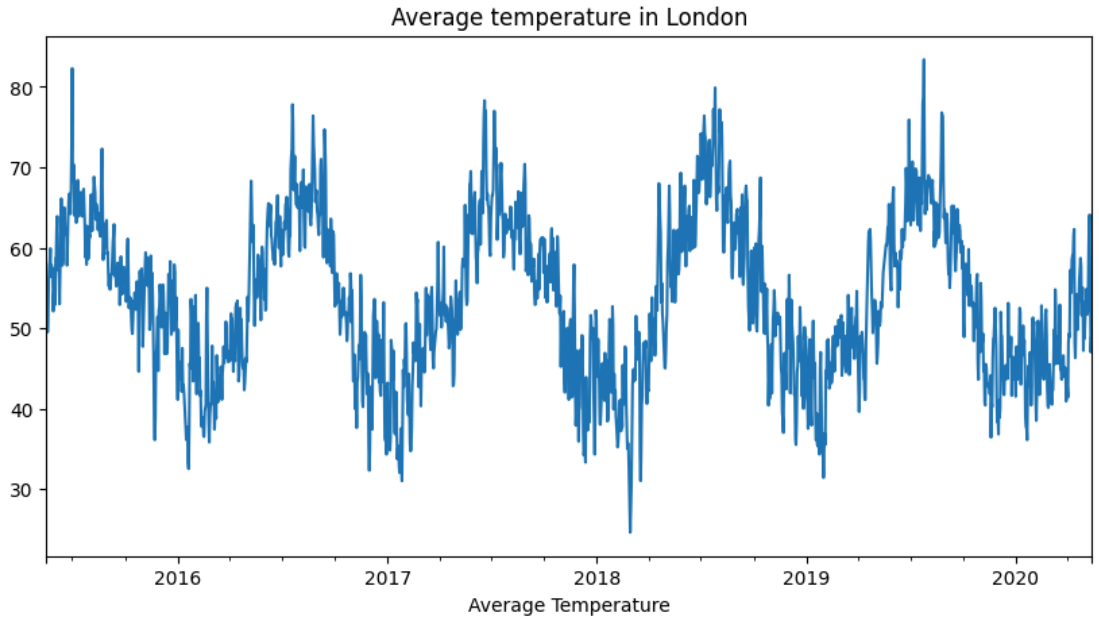
There is something important to note. The presence of a cycle structure does not mean seasonality. For this cycle to be seasonal it needs to be consistently repeated at the same frequency. Otherwise, it is called a cycle, not a season.

Time series components

Time series data can be decomposed into different components:

- **Trend or secular element**: it is the long-term direction of the data
- **Seasonal element**: systematic, calendar-related variations. Consistently repeated at the same frequency.
- **Cyclical element**: periodical, but not seasonal variations.
- **Residual or irregular element**: unsystematic, short-term fluctuations. It is also called noise.

Example of seasonal time series data: Average temperature in London since 2016



Decomposition models

Time series data can include any of these components. However, not all of them include them in the same way. Let's find out the most common models that take into account these elements. For simplicity, we will ignore the cyclical element.

Additive Model

These models assume the observed time series is the sum of its elements:

$$y(t)=\text{Trend}+\text{Seasonality}+\text{Residual}$$

This model considers that the magnitudes of the seasonal and residual elements are independent of the trend.

Multiplicative Model

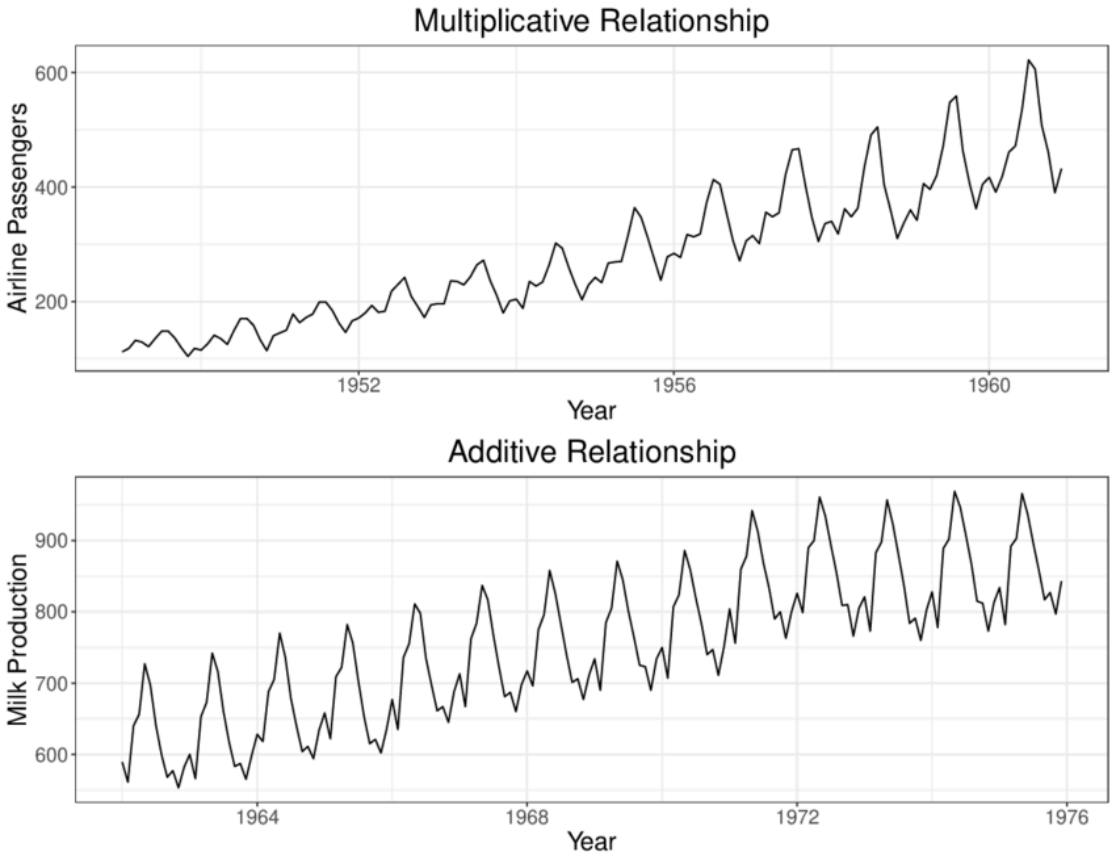
These models assume the observed time series is the product of its elements:

$$y(t)=\text{Trend}\times\text{Seasonality}\times\text{Residual}$$

This model implies that the seasonality and residuals are dependent on the trend.

This model can be transformed into an additive model by applying logarithmic transformations:

$$\log(y(t))=\log(\text{Trend})+\log(\text{Seasonality})+\log(\text{Residual})$$

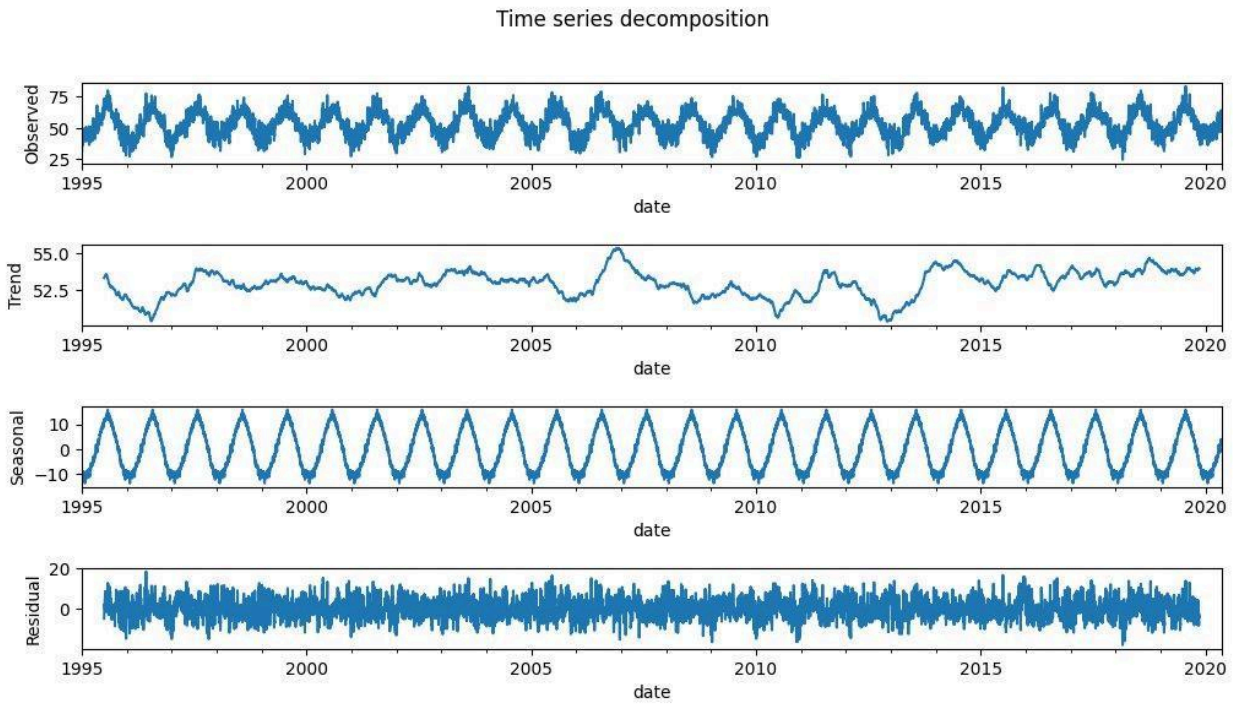


Additive and multiplicative decomposition models

Other models

In addition to the previous two models, there are other models that are also used to decompose time series data:

- Pseudo-additive model
- Exponential smoothing
- Locally Estimated Scatterplot Smoothing (LOESS)
- Frequency-based methods



Time Series additive decomposition for the temperature in London since 1995

SARIMA model

If our data exhibits seasonality, we will require a Seasonal ARIMA model or so-called SARIMA. The SARIMA model is an extension of the ARIMA model to explicitly model the Seasonal component in univariate data.

A **SARIMA(p,d,q)(P,D,Q)m** model is defined by seven parameters. Three come from the ARIMA part and an additional four parameters characterize the modelling of the seasonal element:

Trend

- *p* : trend autoregressive order
- *d* : trend difference order
- *q* : trend moving average order

Seasonality

- *P* : seasonal autoregressive order
- *D* : seasonal difference order
- *Q* : seasonal moving average order
- *m* : number of timesteps for a seasonal period

Selection of parameters

The first parameter to choose is ***m***. This must be chosen by inspecting the initial data and having some knowledge of what represents.

After knowing the number of timesteps of a single seasonal period, the data will be decomposed.

p and ***q*** will be selected by looking at the trend element using the ACF and PACF plots. ***d*** will be chosen so it makes the trend stationary.

In the same way, ACF and PACF plots are used to estimate the values of ***P*** and ***Q***, by looking at the correlation of seasonal lags. ***D*** will be chosen to make a period of the seasonal element stationary.

Points to consider:

- Generally total order of differencing (d+D) should be not more than two.
- Even though we derive p and P values from PACF plots and q and Q values from ACF plots, we have to overfit, check residues, check performance. Model building is an art which requires us to consider various points before shortlisting the models.
- AIC should be used to compare the models with the same order of differencing

Limitations

The limitation of the SARIMA models is that they can be used only with single seasonality. For example, there could be cases in which we could have seasonality during the day, during the week and also during the year. There are other models able to deal with that, for instance: TBABS.

Practical case

Let's see a practical example now. We will try to forecast the maximum monthly temperature in Madrid in 2022.

We need to first import some basic libraries.

Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import datetime
```

To get the data we will use the meteostat library:

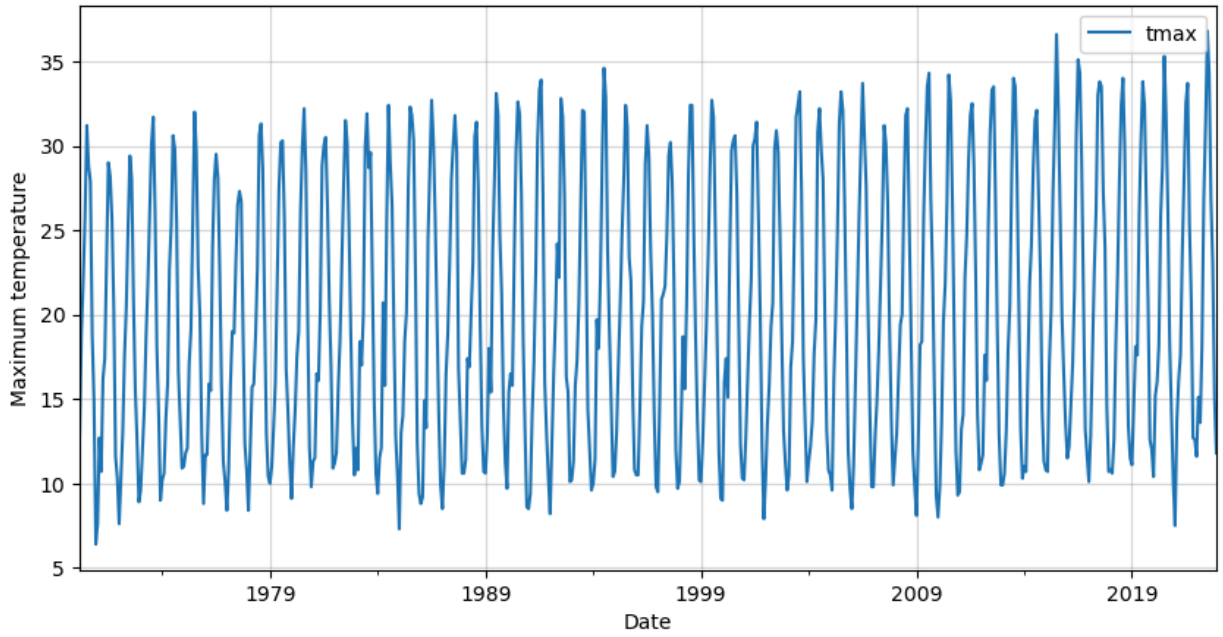
Python

```python
# Install library
!pip install meteostat

# Import Meteostat library and dependencies
from meteostat import Point, Monthly

# Set time period
start = datetime.datetime(1970, 3, 1)
end = datetime.datetime(2022, 12, 31)

# Create Point for Madrid
location = Point(40.416775, -3.703790, 657)

# Get daily data for March 2023
df_madrid = Monthly(location, start, end)
df_madrid = df_madrid.fetch()
```

Let's remove unwanted columns and keep only the maximum temperature.

Python

```python
# Keep only the maximum temperature
df_temp_max = df_madrid[['tmax']]

# Visualize the data
df_temp_max.plot(figsize=(10,5))
plt.grid(alpha=0.5)
plt.xlabel('Date')
plt.ylabel('Maximum temperature')
plt.show()
```



We can clearly see a seasonal pattern in our data. This is to expect as we are dealing with monthly temperatures.

We need to make sure that there are no missing values. If there were, we would need to handle them.

Python

```
# Check for missing values
df_temp_max.isna().sum()
```

```
tmax    0


dtype: int64
```

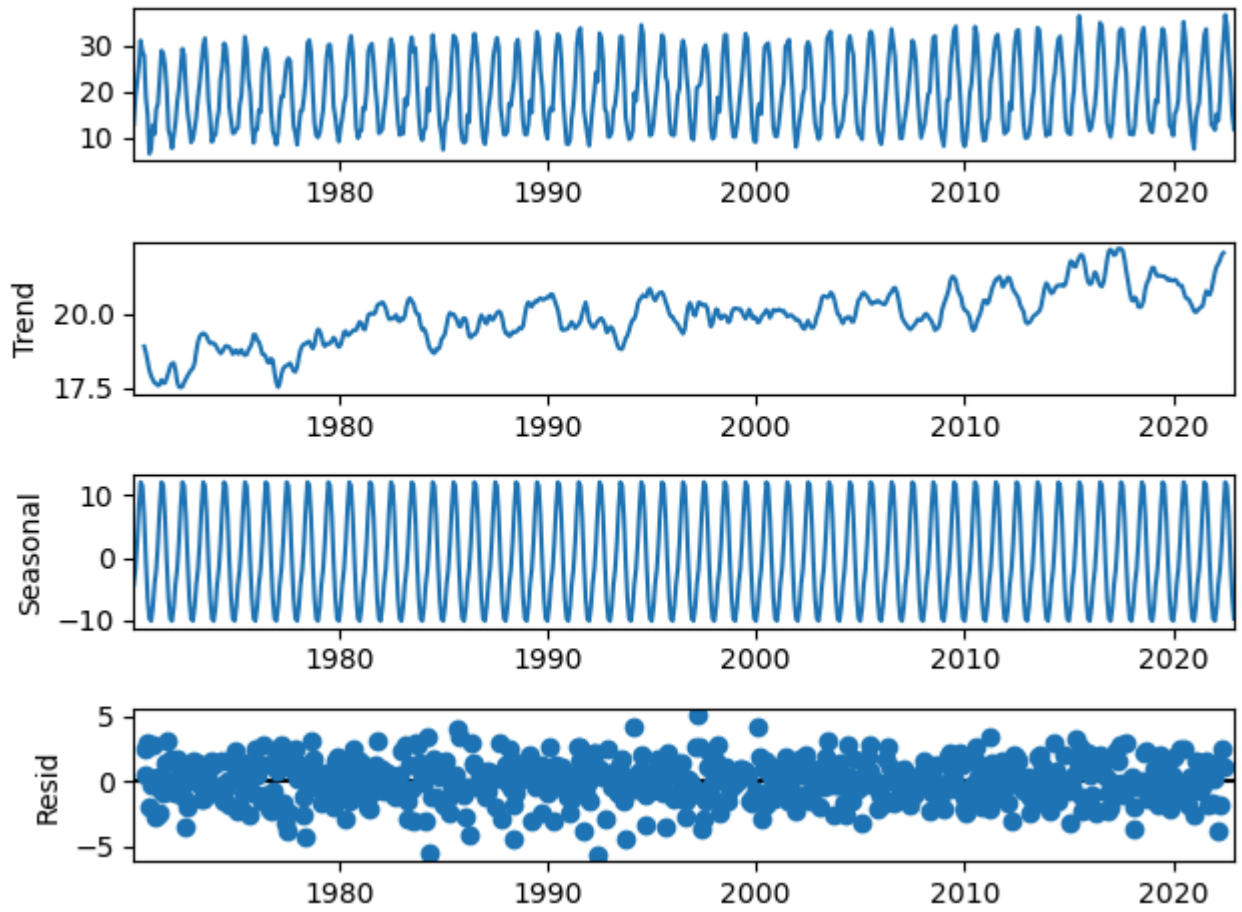
Fortunately for us, there are no missing values in our data.

Let's now proceed to decompose our data, to verify the presence of a seasonal component. We can use the `seasonal_decompose` function of the `statsmodels` library.

We will assume a seasonal period of 12 since we have monthly data. Also, there is nothing in the data that makes us suspect a multiplicative model, the variance is not either increasing or decreasing significantly. Therefore, we will use an additive model.

Python

```
# Import statsmodels library
import statsmodels.api as sm

# Perform seasonal decomposition
decomposition = sm.tsa.seasonal_decompose(df_temp_max,
                                          model='additive',
                                          period=12)

# Extract the decomposed components
trend = decomposition.trend
seasonal = decomposition.seasonal
residual = decomposition.resid

# Plot the decomposed components
decomposition.plot()
plt.show()
```



This confirms our suspicion of seasonality in the data. Therefore, we will use a SARIMA model to forecast the future maximum temperatures in Madrid.

Check for stationarity

We could check stationarity, however, this is not necessary as we will use a SARIMA model.

However, it could be useful to estimate the `d` parameter and speed up the process.

We can use the Augmented Dickey–Fuller test to see whether there is a unit root in our time series data.

Python

```python
from statsmodels.tsa.stattools import adfuller

# Perform Augmented Dickey-Fuller test
result = adfuller(df_temp_max)

# Extract and print the test statistics and p-value
test_statistic = result[0]
p_value = result[1]
print(f"Test Statistic: {test_statistic}")
print(f"P-value: {p_value}")
```

```
Test Statistic: -2.971872590243202

P-value: 0.03760595188739091
```

We got a p-value lower than 0.05, therefore, we can assume that our data is stationary.

Since the p-value is in the limit, let's also use a Kwiatkowski–Phillips–Schmidt–Shin (KPSS) test to verify our assumption. This test examines the presence of a trend or other forms of non-stationarity.

Python

```python
from statsmodels.tsa.stattools import kpss

# Perform KPSS test
result = kpss(df_temp_max)

# Extract and print the test statistic and p-value
test_statistic = result[0]
p_value = result[1]
print(f"Test Statistic: {test_statistic}")
print(f"P-value: {p_value}")
```

```
Test Statistic: 0.23003042556646808

P-value: 0.1
```

In this case, we are interested in a p-value larger than 0.05. This is because of how the hypotheses are defined. The p-value is larger than 0.05 and therefore it confirms the stationarity of the data.

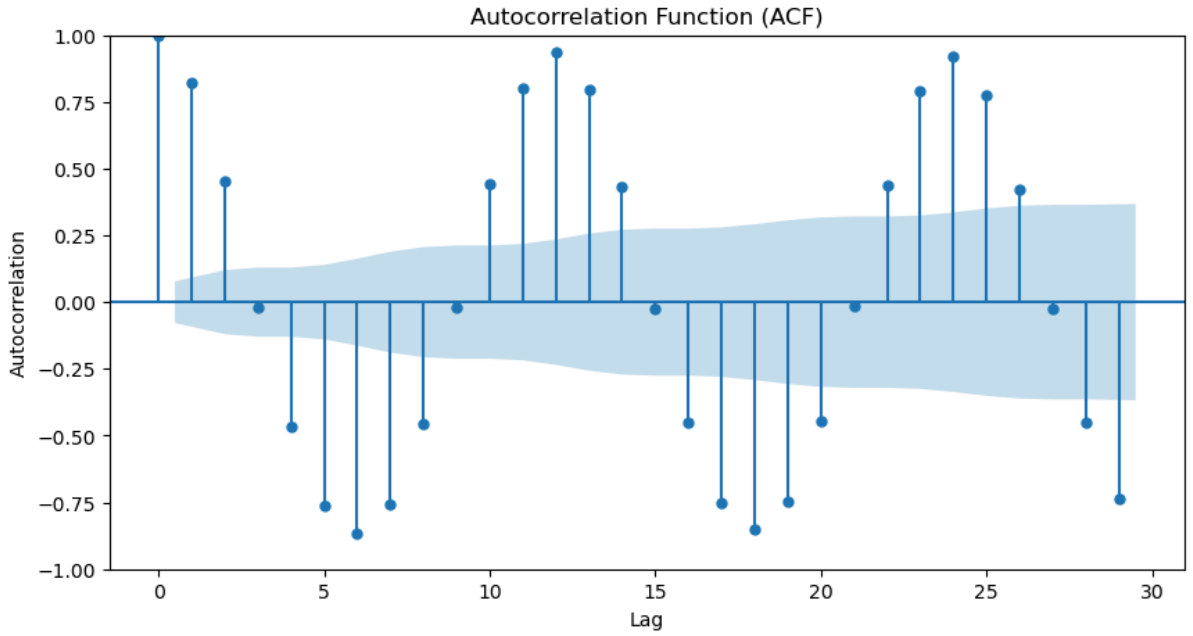
This is telling us that the `d` parameter will be 0, or what is the same, that no differencing will be necessary.

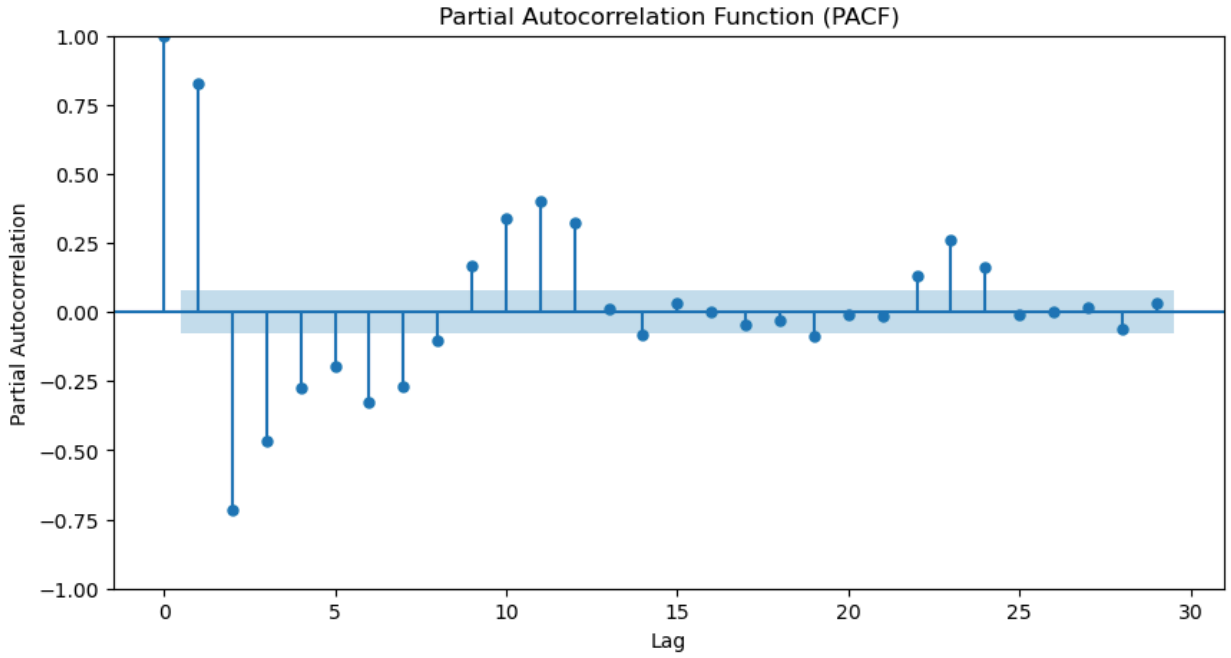
For the sake of completeness, we could check the ACF and PACF plots. This can confirm the presence of seasonality and the seasonal period.

Python

```python
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# Plot ACF
fig, ax = plt.subplots(figsize=(10, 5))
plot_acf(df_temp_max, ax=ax)
plt.xlabel('Lag')
plt.ylabel('Autocorrelation')
plt.title('Autocorrelation Function (ACF)')
plt.show()

# Plot PACF
fig, ax = plt.subplots(figsize=(10, 5))
plot_pacf(df_temp_max, ax=ax)
plt.xlabel('Lag')
```

```
plt.ylabel('Partial Autocorrelation')
plt.title('Partial Autocorrelation Function (PACF)')
plt.show()
```





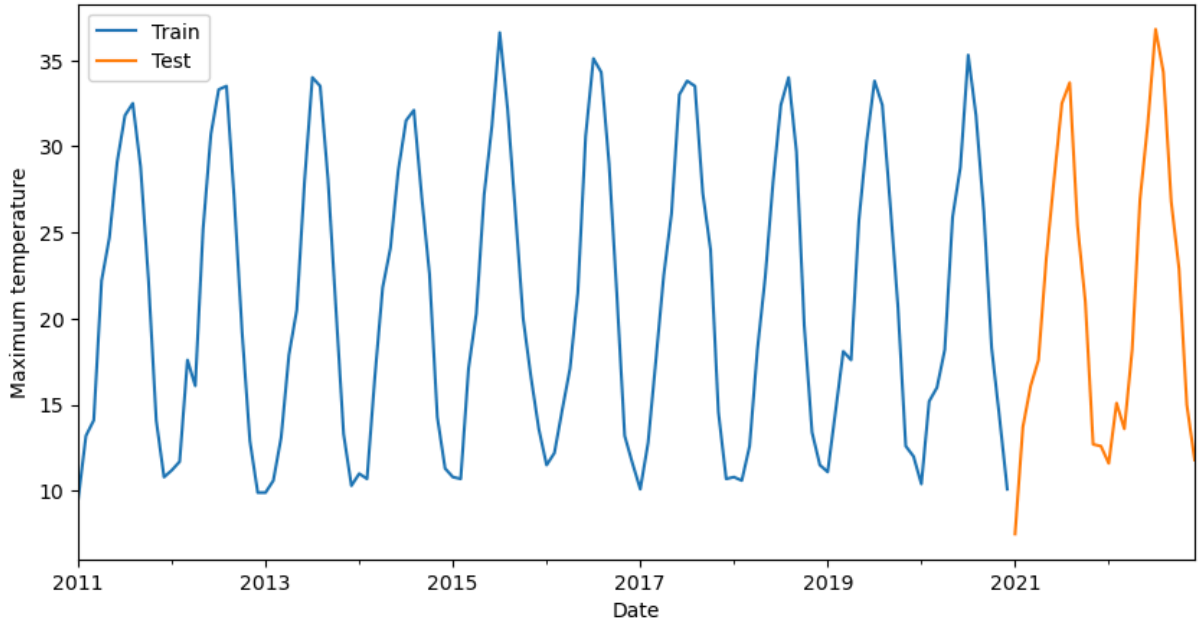
From the ACF and PACF, we can see a strong seasonality present in our data. From the ACF we can verify that the seasonal period is 12 as is expected due to our monthly data and a year having 12 months.

Before training our model we need to split our data into training and testing. For example, we could leave 2 years for forecasting and the remaining ones for training our model. This all depends on your specific objective.

Python

```
# Split into training and testing
df_train = df_temp_max.loc[:'2020']
df_test = df_temp_max.loc['2021':]
```

```
# Plot the last 10 years of training data and the 2 of testing
ax = df_train[-12*10:].plot(figsize=(10, 5))
df_test.plot(ax=ax)
plt.legend(['Train', 'Test'])
plt.xlabel('Date')
plt.ylabel('Maximum temperature')
plt.show()
```



Once we have our training data ready we can proceed to train the model.

A SARIMA model is defined by 7 parameters as we mentioned before: (p, d, q)(P, D, Q, m) We know that our model does not need differencing to be stationary, therefore `d` will most likely be 0. Also, we know the seasonal period of our data is 12, so `m` will be 12.

Optimal model

We need to find the optimal model. What does this mean? The model must have enough prediction power while being as simple as possible to prevent overfitting the data.

For doing this we will iterate with all the reasonable combinations of parameters. This is called "**grid search**". We need to fit each SARIMA model with the chosen parameter values to the training data and compare an evaluation metric.

Generally, the metric chosen is the AIC. AIC stands for Akaike Information Criterion. It is a statistical measure used to evaluate the quality of a model by balancing its goodness of fit with its complexity (number of parameters). The AIC is commonly used in model selection, including the selection of SARIMA models.

The AIC takes into account both the model's ability to fit the data and the number of parameters used in the model. The formula for calculating AIC is as follows:

$$AIC = -2 \times \text{log-likelihood} + 2 \times k$$

where:

- ***log-likelihood*** represents the maximized log-likelihood of the model
- ***k*** is the number of parameters in the model

During the grid search, the AIC will be calculated for each model, the model with the lowest AIC value will indicate the best balance between accuracy and simplicity for your particular time series data.

Also, we need to bear in mind that we must keep only those models that converge. To ensure this we could just check the z-values, and ignore any models with any of them with an infinite value.

Python

```python
import itertools
import math

# Define the range of values for p, d, q, P, D, Q, and m
p_values = range(0, 3)  # Autoregressive order
d_values = [0]          # Differencing order
q_values = range(0, 3)  # Moving average order
P_values = range(0, 2)  # Seasonal autoregressive order
D_values = range(0, 1)  # Seasonal differencing order
Q_values = range(0, 2)  # Seasonal moving average order
m_values = [12]         # Seasonal period

# Create all possible combinations of SARIMA parameters
param_combinations = list(itertools.product(p_values,
                                            d_values,
                                            q_values,
                                            P_values,
                                            D_values,
                                            Q_values,
                                            m_values))

# Initialize AIC with a large value
best_aic = float("inf")
best_params = None

# Perform grid search
for params in param_combinations:
    order = params[:3]
    seasonal_order = params[3:]

    try:
        model = sm.tsa.SARIMAX(df_train,
                               order=order,
                               easonal_order=seasonal_order)
        result = model.fit(disp=False)
        aic = result.aic

        # Ensure the convergence of the model
```

```
        if not math.isinf(result.zvalues.mean()):
            print(order, seasonal_order, aic)

            if aic < best_aic:
                best_aic = aic
                best_params = params

        else:
            print(order, seasonal_order, 'not converged')

    except:
        continue

# Print the best parameters and AIC
print("Best Parameters:", best_params)
print("Best AIC:", best_aic)
```

```
Best Parameters: (2, 0, 1, 1, 0, 1, 12)


Best AIC: 2434.0270656007365
```

SARIMA model

Let's then fit the model with the optimal parameters and check its summary.

Python

```
model = sm.tsa.SARIMAX(df_train,
                       order=best_params[:3],
                       seasonal_order=best_params[3:])
result = model.fit(disp=False)

# Show the summary
result.summary()
```

SARIMAX Results

Dep. Variable:	tmax	**No. Observations:**	610
Model:	SARIMAX(2, 0, 1)x(1, 0, 1, 12)	**Log Likelihood**	-1211.014
Date:	Wed, 14 Jun 2023	**AIC**	2434.027
Time:	13:47:42	**BIC**	2460.508
Sample:	03-01-1970	**HQIC**	2444.328
	- 12-01-2020		
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	1.2294	0.028	44.614	0.000	1.175	1.283
ar.L2	-0.2310	0.027	-8.563	0.000	-0.284	-0.178
ma.L1	-0.9843	0.013	-77.359	0.000	-1.009	-0.959
ar.S.L12	0.9994	0.000	2630.607	0.000	0.999	1.000
ma.S.L12	-0.8812	0.022	-40.736	0.000	-0.924	-0.839
sigma2	2.8581	0.110	25.889	0.000	2.642	3.075

Ljung-Box (L1) (Q):	0.03	**Jarque-Bera (JB):**	3.89
Prob(Q):	0.86	**Prob(JB):**	0.14
Heteroskedasticity (H):	0.76	**Skew:**	-0.16
Prob(H) (two-sided):	0.06	**Kurtosis:**	3.21

Let's inspect the summary. We will focus on the coefficients part. Our SARIMA model has the following parameter estimates:

- The autoregressive terms (ar.L1 and ar.L2) have estimated coefficients of 1.2218 and -0.2248, respectively.
- The moving average term (ma.L1) has an estimated coefficient of -0.9846.
- The seasonal autoregressive term (ar.S.L12) has an estimated coefficient of 0.9988.
- The seasonal moving average term (ma.S.L12) has an estimated coefficient of -0.8444.
- The estimated variance of the error term (sigma2) is 2.9183.

The "P>|z|" column provides the p-value associated with each coefficient, for all cases they are lower than 0.05, our significance level, which means that all of them are statistically significant.

When you run a SARIMA (Seasonal AutoRegressive Integrated Moving Average) model in Python, typically using the statsmodels library, you get an output table that summarizes the results of the model fitting. Here's a breakdown of the key components you might see in the output table:

1. Model Summary

This section provides an overview of the model and the data used.

- Dep. Variable: The dependent variable (the variable you're trying to predict).No. Observations: The number of observations in the dataset.
- Model: The type of model used (e.g., SARIMA).
- Date: The date when the model was run.
- Time: The time when the model was run.
- AIC: Akaike Information Criterion, a measure of the model's quality (lower is better).
- BIC: Bayesian Information Criterion, another measure of model quality (lower is better).
- HQIC: Hannan-Quinn Information Criterion, yet another measure of model quality (lower is better).

2. Coefficients Table

This section lists the estimated coefficients for the model parameters.

- coef: The estimated value of the coefficient.
- std err: The standard error of the coefficient estimate.
- z: The z-score, which is the coefficient divided by its standard error.
- P>|z|: The p-value, which indicates the significance of the coefficient (a lower value suggests higher significance).
- [0.025, 0.975]: The 95% confidence interval for the coefficient.

3. Diagnostics

This section provides diagnostic information to assess the model fit.

- Ljung-Box (Q): A test statistic for checking the independence of residuals.
- Jarque-Bera (JB): A test statistic for checking the normality of residuals.
- Heteroskedasticity (H): A test statistic for checking the homoscedasticity (constant variance) of residuals.

- Skew: A measure of the asymmetry of the residuals.
- Kurtosis: A measure of the "tailedness" of the residuals.

Example Output Table Interpretation

SARIMAX Results

==========

Dep. Variable: y No. Observations: 100

Model: SARIMAX(1, 1, 1)x(1, 1, 1, 12) Log Likelihood -200.123

Date: Thu, 20 Mar 2025 AIC 410.246

Time: 12:34:56 BIC 420.567

Sample: 0 HQIC 414.123

- 100

==========

	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	0.5000	0.100	5.000	0.000	0.300	0.700
ma.L1	-0.3000	0.150	-2.000	0.045	-0.600	-0.000
ar.S.L12	0.2000	0.050	4.000	0.001	0.100	0.300
ma.S.L12	-0.1000	0.070	-1.429	0.153	-0.240	0.040
sigma2	1.0000	0.200	5.000	0.000	0.600	1.400

==========

Ljung-Box (Q):	0.123	Jarque-Bera (JB):	1.234
Prob(Q):	0.725	Prob(JB):	0.540
Heteroskedasticity (H):	1.000	Skew:	0.123
Prob(H) (two-sided):	0.999	Kurtosis:	3.000

Key Points to Note:

Coefficients: Look at the coef column to understand the estimated values for each parameter. Significant coefficients (low p-values) indicate important predictors.

Model Fit: Lower AIC, BIC, and HQIC values suggest a better-fitting model.

Diagnostics: Check the Ljung-Box, Jarque-Bera, and Heteroskedasticity tests to ensure the residuals meet the assumptions of the model.

ARIMA vs SARIMA

So what’s the difference between ARIMA and SARIMA and why would we use it?

Well, as you might already know, seasonality is an important factor in forecasting. As the seasons change, they have a huge impact on your data.

For example

Christmas sales in a store vs January.

Unfortunately, ARIMA doesn't have a seasonal component, which is why SARIMA was developed.

It’s not the only difference though.

Remember the p,d, and q components from the ARIMA model?

To build a SARIMA model, you need to add both seasonal components and an extra differencing component.

- The seasonal components are the seasonal autoregressive (SAR) and seasonal moving average (SMA) components
- While the extra differencing component is for the seasonal component and is denoted by D

Now, to find the optimal component values, you will also need to use the Cross-Validation and Parameter Tuning. (Granted, a shortcut is the auto_arima function, but more on that in a second).

tl;dr

With the SARIMA model, you can capture both the non-seasonal and seasonal patterns in your data and build a forecasting model.

It's like playing detective with your data, hunting for the optimal values of p, d, q, P, D, and Q to solve the mystery of forecasting your time series.